Verbetersleutel – Escape room: begrijp je de code?

Bij elke vraag vind je het antwoord en een uitleg. Alle antwoorden werden gecontroleerd door de code uit te voeren.

- Bij een foutmelding volstaat het antwoord **fout**: welk soort fout hoeft niet vermeld te worden.
- De regelnummers verwijzen naar de code hieronder.

Ter referentie, de code van het spel:

```
1  # ESCAPE ROOM: Opgesloten in de school
2  # Voorbeeld met 2 kamers. Een groepje maakt er zelf 4 à 5.
3  #
4  # Basisregels:
5  # - een functie krijgt alles wat ze nodig heeft via parameters;
6  # - variabelen die in een functie ontstaan, zijn lokaal: ze bestaan enkel in die functie;
7  # - een functie geeft haar resultaat terug met return (een getal, een tekst, True/False);
8  # - de lijst inventaris wordt enkel in het hoofdprogramma gebruikt en aangepast.
9
10
11 def kamer_klaslokaal():
12     """Eerste kamer: geeft het gevonden voorwerp terug, of een lege tekst als je niets vindt."""
13     print("\n--- HET KLASLOKAAL ---")
14     print("Je zit opgesloten in lokaal B12. Je ziet een bureau en een kast.")
15     keuze = input("Wat onderzoek je? (bureau/kast) ").lower()
16     if keuze == "bureau":
17         print("In de lade ligt een sleutel! Je steekt hem in je zak.")
18         return "sleutel"
19     elif keuze == "kast":
20         print("Enkel oude cursussen wiskunde... Je vindt niets.")
21     else:
22         print("Dat ken ik niet. Je verliest tijd.")
23     return ""
24
25
26 def kamer_gang(heeft_sleutel):
27     """Tweede kamer: deur met een cijferslot. Geeft True terug als de speler ontsnapt."""
28     print("\n--- DE GANG ---")
29     if not heeft_sleutel:
30         print("De deur van het lokaal zit op slot. Je hebt een sleutel nodig!")
31         return False
32
33     print("Je opent de deur. Aan de uitgang hangt een cijferslot met een briefje:")
34     print("'De code is de schuine zijde van een rechthoekige driehoek met zijden 20 en 21.'")
35     pogingen = 0
36     while pogingen < 3:
37         poging = int(input("Code? "))
38         pogingen += 1
39         if poging == schuine_zijde(20, 21):
40             print("KLIK! De deur gaat open.")
41             return True
42         print(f"Fout! Nog {3 - pogingen} poging(en).")
43     print("Het alarm gaat af...")
44     return False
45
46
47 def schuine_zijde(a, b):
48     """Schuine zijde van een rechthoekige driehoek met rechthoekszijden a en b."""
49     return int((a ** 2 + b ** 2) ** 0.5)
50
51
52 # ----- HOOFDPROGRAMMA -----
53 naam = input("Hoe heet je? ")
54 inventaris = []
55
56 voorwerp = kamer_klaslokaal()
57 if voorwerp != "":
58     inventaris.append(voorwerp)
59
60 ontsnapt = kamer_gang("sleutel" in inventaris)
61
```

```
62  if ontsnapt:
63      print(f"\nProficiat {naam}, je bent ontsnapt met: {inventaris}")
64  else:
65      print(f"\nHelaas {naam}, je zit nog altijd vast. GAME OVER")
66
```

Deel A: Het verloop van het programma volgen

Vraag 1

Python leest de code van boven naar onder.

- Wat is de **eerste regel** die de speler iets laat zien of iets vraagt? Wat verschijnt er dan op het scherm?
- De regel `print("\n--- HET KLASLOKAAL ---")` staat veel hoger in de code. Waarom verschijnt die toch niet als eerste? Wat doet Python wanneer het een regel `def ...` tegenkomt?

Antwoord:

- De eerste regel die de speler iets toont, is regel 53: `naam = input("Hoe heet je? ")`. Op het scherm verschijnt `Hoe heet je?`.
- Bij een regel `def ...` wordt de definitie van de functie opgeslagen (in het geheugen van de kernel), maar de functie wordt **niet** onmiddellijk opgeroepen. De code in `kamer_klaslokaal` wordt pas uitgevoerd bij de oproep in regel 56: `voorwerp = kamer_klaslokaal()`.

Vraag 2

De speler typt: `Ann`, dan `KAST` (in hoofdletters). Wat zijn **alle regels** die het programma op het scherm toont? Leg uit.

Antwoord:

```
Hoe heet je? Ann

--- HET KLASLOKAAL ---
Je zit opgesloten in lokaal B12. Je ziet een bureau en een kast.
Wat onderzoek je? (bureau/kast) KAST
Enkel oude cursussen wiskunde... Je vindt niets.

--- DE GANG ---
De deur van het lokaal zit op slot. Je hebt een sleutel nodig!

Helaas Ann, je zit nog altijd vast. GAME OVER
```

- `Ann` en `KAST` zijn wat de speler typt; ze verschijnen achter de vraag van `input`.
- De lege regels komen van `\n` aan het begin van een tekst: dat is een nieuwe regel.
- `.lower()` zet `"KAST"` om naar `"kast"`, dus de keuze wordt herkend: `Enkel oude cursussen wiskunde... Je vindt niets.`
- In de `elif` staat geen `return`, dus de functie loopt door tot regel 23: `return ""` → `voorwerp = ""` → er wordt niets toegevoegd: `inventaris = []`.
- `"sleutel" in inventaris` is False → `kamer_gang(False)` toont `--- DE GANG ---` en `De deur van het lokaal zit op slot...` en geeft `False` terug.
- `ontsnapt = False` → de `else` in het hoofdprogramma wordt uitgevoerd.

Vraag 3

De speler typt: `Lena`, `bureau`, `41`, `29`. In welke **variabele** wordt elke ingetypte waarde opgeslagen, en wat is het **type** van die variabele? Vermeld ook het regelnummer.

Antwoord:

invoer	variabele	regel	type
`Lena`	`naam` (hoofdprogramma)	53	str
`bureau`	`keuze` (lokaal in `kamer_klaslokaal`)	15	str
`41`	`poging` (lokaal in `kamer_gang`)	37	int
`29`	`poging` (lokaal in `kamer_gang`)	37	int

- `input()` geeft altijd tekst (str). `naam` en `keuze` blijven dus str; bij `keuze` wordt de tekst nog omgezet naar kleine letters met `.lower()`.
- Bij `poging` zet `int( )` de tekst om naar een geheel getal → int.
- `41` en `29` komen in **dezelfde** variabele `poging` terecht: bij de tweede doorloop van de lus wordt 41 overschreven door 29.

Vraag 4

De speler typt: `Lena`, `bureau`, `41`, `29`. Vul de tabel in voor de `while`-lus in `kamer_gang` (regel 36 tot 42). In de laatste rij vul je het **type** in.

Antwoord:

doorloop	pogingen bij de controle `pogingen < 3` (regel 36)	poging (regel 37)	pogingen na `pogingen += 1` (regel 38)	`poging == schuine_zijde(20, 21)`? (regel 39)	uitvoer
1	0 → True	41	1	41 == 29 → False	`Fout! Nog 2 poging(en).`
2	1 → True	29	2	29 == 29 → True	`KLIK! De deur gaat open.`
type	int	int	int	bool	–

- De volgorde in de lus: eerst `input` (regel 37), dan `pogingen += 1` (regel 38), pas daarna de vergelijking (regel 39). Daarom is `pogingen` al 1 bij de eerste foutboodschap: 3 − 1 = 2.
- In doorloop 2 beëindigt `return True` (regel 41) de functie meteen: er komt geen derde doorloop.
- Daarna toont het hoofdprogramma: `Proficiat Lena, je bent ontsnapt met: ['sleutel']`

Vraag 5

Bij welke invoer verschijnt `Het alarm gaat af...`? Hoeveel keer kan `Code?` maximaal verschijnen?

Antwoord:

`Het alarm gaat af...` verschijnt bij deze invoer:

1. een willekeurige naam;
2. `bureau`, in grote of kleine letters (door `.lower()`);
3. 3 keer een getal dat verschilt van 29.

`Code?` kan dus maximaal **3 keer** verschijnen.

Vraag 6

Wat geeft `schuine_zijde(20, 21)` terug? Werk stap voor stap uit. Wat geeft `schuine_zijde(5, 5)` terug?

Antwoord:

stap	`schuine_zijde(20, 21)`	`schuine_zijde(5, 5)`
`a ** 2 + b ** 2`	400 + 441 = 841	25 + 25 = 50
`(...) ** 0.5` (vierkantswortel)	29.0	7.0710678...
`int(...)`	**29**	**7**

- `** 0.5` is hetzelfde als de vierkantswortel nemen (stelling van Pythagoras). Het resultaat is een float.
- `int()` **kapt af** (rondt niet af): 7.07... wordt 7.

Deel B: Lokale variabelen, parameters en return

Vraag 7

Welke variabelen zijn **lokaal** in de functie `kamer_gang`? Welke variabelen horen bij het **hoofdprogramma**? Geef bij elke variabele ook het **type**.

Antwoord:

variabele	waar?	type
`heeft_sleutel` (parameter)	lokaal in `kamer_gang`	bool
`pogingen`	lokaal in `kamer_gang`	int
`poging`	lokaal in `kamer_gang`	int
`naam`	hoofdprogramma	str
`inventaris`	hoofdprogramma	list
`voorwerp`	hoofdprogramma	str
`ontsnapt`	hoofdprogramma	bool

- Een parameter is ook een lokale variabele.
- (Niet gevraagd, maar ter info: `keuze` is lokaal in `kamer_klaslokaal`; `a` en `b` zijn lokaal in `schuine_zijde`.)

Vraag 8

Je voegt helemaal onderaan het programma deze regel toe: `print(pogingen)`. De speler typt `Jo`, `bureau`, `29`. Wat gebeurt er?

Antwoord:

Het spel verloopt eerst gewoon (de deur gaat open, `Proficiat Jo, je bent ontsnapt met: ['sleutel']`). De nieuwe regel `print(pogingen)` geeft daarna een **fout**.

- `pogingen` is een **lokale** variabele van `kamer_gang`: ze bestaat enkel zolang die functie bezig is.
- Na `return True` is de functie afgelopen en verdwijnt `pogingen`. In het hoofdprogramma bestaat er geen variabele met die naam.

Vraag 9

De speler kiest `kast`. Welke waarde krijgt de parameter `heeft_sleutel` in `kamer_gang`? Hoe ontstaat die waarde?

Antwoord:

`heeft_sleutel` krijgt de waarde **`False`**.

- Regel 56: `voorwerp = kamer_klaslokaal()`. De functie `kamer_klaslokaal` wordt uitgevoerd. De gebruiker voert `kast` in, dus de variabele `keuze = "kast"` (regel 15). Er wordt iets geprint (`Enkel oude cursussen wiskunde... Je vindt niets.`) en er wordt een lege string `""` geretourneerd (regel 23) en opgeslagen in de variabele `voorwerp`.
- Regel 57: `voorwerp != ""` is False, dus regel 58 wordt niet uitgevoerd. `inventaris` blijft een lege lijst.

- Regel 60: `ontsnapt = kamer_gang("sleutel" in inventaris)`. `inventaris` is een lege lijst, de string `"sleutel"` zit dus niet in de lijst. `"sleutel" in inventaris` is dus een boolean die gelijk is aan `False`, en de parameter `heeft_sleutel` krijgt als waarde `False` mee.

Vraag 10

In `kamer_gang` staat **drie** keer `return`. Wanneer wordt elke `return` uitgevoerd, en wat geeft de functie dan terug? Waarom verschijnt de zin `Je opent de deur...` niet als de speler geen sleutel heeft?

Antwoord:

`return`	**wanneer?**	**geeft terug**
1e: regel 31, in `if not heeft_sleutel:`	de speler heeft geen sleutel	`False`
2e: regel 41, in de `while`-lus	de speler heeft een sleutel en typt de juiste code (29)	`True`
3e: regel 44, helemaal onderaan	de speler heeft een sleutel maar typt 3 keer een foute code	`False`

- `return` **beëindigt de functie onmiddellijk**: alle regels eronder worden niet meer uitgevoerd.
- Zonder sleutel wordt de eerste `return False` uitgevoerd. De functie stopt daar, dus `print("Je opent de deur...")` wordt nooit bereikt. Een `else` is daarom niet nodig.

Vraag 11

★ Je verwijdert de laatste regel `return ""` uit `kamer_klaslokaal`. De speler typt `Bo`, `kast`. Verandert er iets aan wat op het scherm verschijnt? Welke waarde krijgen `voorwerp` en `inventaris`?

Antwoord:

Op het scherm verandert er **niets**:

```
Enkel oude cursussen wiskunde... Je vindt niets.

--- DE GANG ---
De deur van het lokaal zit op slot. Je hebt een sleutel nodig!

Helaas Bo, je zit nog altijd vast. GAME OVER
```

Maar binnenin wel:

- Een functie die eindigt zonder `return`, geeft automatisch **`None`** terug ("niets"). Dus `voorwerp = None`.
- `None != ""` is **True** (`None` is geen lege tekst!) → `inventaris.append(None)` → `inventaris = [None]`.
- `"sleutel" in [None]` is nog altijd False, dus het spel verloopt hetzelfde. Omdat de inventaris enkel bij een ontsnapping getoond wordt, zie je het verschil niet.
- Antwoord: `voorwerp = None`, `inventaris = [None]`.

Vraag 12

De functie `schuine_zijde` wordt gebruikt in `kamer_gang`, maar staat er **onder**. Toch geeft dat geen fout. Waarom niet? Wat zou er gebeuren als het hoofdprogramma helemaal **bovenaan** stond?

Antwoord:

- Bij het lezen van `def kamer_gang` wordt de inhoud nog niet uitgevoerd (zie vraag 1). `schuine_zijde(20, 21)` wordt pas echt opgeroepen wanneer `kamer_gang` loopt, en dat gebeurt pas in het hoofdprogramma onderaan. Op dat moment zijn **alle** functies al gedefinieerd, dus `schuine_zijde` bestaat.
- Wat telt is dus niet waar de functie in de code staat, maar dat ze gedefinieerd is **vóór ze opgeroepen wordt**.
- Staat het hoofdprogramma bovenaan, dan verschijnt eerst nog `Hoe heet je?`. Na de invoer geeft de regel `voorwerp = kamer_klaslokaal()` een **fout**: die functie bestaat op dat moment nog niet.

Deel C: Wat als...? Pas aan

Vraag 13

Wat gebeurt er als je de regel `pogingen += 1` weglaat en de speler (met sleutel) steeds een foute code typt?

Antwoord:

- `pogingen` blijft altijd **0**, dus `pogingen < 3` blijft altijd True → **oneindige lus**: het programma blijft `Code?` vragen.
- Na elke foute code verschijnt telkens `Fout! Nog 3 poging(en).` (3 − 0 = 3).
- `Het alarm gaat af...` verschijnt nooit. De lus stopt pas als de speler de juiste code (29) typt: dan beëindigt `return True` de functie.

Vraag 14

Iemand vergeet `int( )` in de regel `poging = int(input("Code? "))` van `kamer_gang`. Er staat nu dus `poging = input("Code? ")`. De speler heeft de sleutel en typt de juiste code `29`.

- Wat verschijnt er op het scherm na deze invoer?
- Gaat de deur open? Geeft het programma een foutmelding? Leg uit.

Antwoord:

```
Fout! Nog 2 poging(en).
```

- `input()` geeft altijd **tekst** terug: `poging = "29"`.
- `schuine_zijde(20, 21)` geeft het **getal** `29`.
- `"29" == 29` → **False**: een tekst en een getal zijn nooit gelijk voor Python. Dit is **geen foutmelding**, gewoon False.
- De deur gaat dus **nooit** open, ook niet met de juiste code. Na 3 pogingen: `Het alarm gaat af...` en GAME OVER.
- Verbetering: `poging = int(input("Code? "))` (of vergelijken met `str(schuine_zijde(20, 21))`).

Vraag 15

Pas de code aan zodat de speler **5** pogingen krijgt in plaats van 3. Op hoeveel plaatsen moet je iets veranderen?

Antwoord:

Op **2 plaatsen** (regel 36 en regel 42):

```
while pogingen < 5:
    ...
    print(f"Fout! Nog {5 - pogingen} poging(en).")
```

- Wie enkel de `while` aanpast, krijgt wel 5 pogingen, maar een foute boodschap: `Nog -1 poging(en).` enz.
- Tip (uitbreiding): met een variabele `max_pogingen = 5` bovenaan de functie hoef je maar op één plaats iets te veranderen.

Vraag 16

Pas de code aan zodat de speler in de **kast** een `zaklamp` vindt. De zaklamp moet in de inventaris komen. Wat moet je veranderen in de functie? Moet je ook iets veranderen in het hoofdprogramma?

Antwoord:

In de functie, in de `elif`-tak (regel 19–20):

```
    elif keuze == "kast":
        print("Tussen de oude cursussen ligt een zaklamp! Je neemt hem mee.")
        return "zaklamp"
```

- De boodschap aanpassen en vooral **`return "zaklamp"`** toevoegen. Zonder `return` komt de functie bij `return ""` en wordt er niets gevonden.
- In het hoofdprogramma hoeft er **niets** te veranderen: `if voorwerp != "":` voegt elk teruggegeven voorwerp toe, dus ook `"zaklamp"` → `inventaris = ['zaklamp']`.
- De functie past zelf de inventaris niet aan (basisregel): ze geeft enkel het voorwerp terug.
- Let op: met de zaklamp kan de speler nog altijd niet ontsnappen, want `kamer_gang` kijkt enkel naar de sleutel.